Conversão de EER para Relacional e Introdução a SQL

Banco de Dados

Prof. Dr. Carlos Melo

carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Mapeamento EER para Modelo Relacional

- Existem 4 opções principais para converter uma hierarquia (Superclasse e Subclasses) para o modelo relacional:
  - 1: Múltiplas Relações (Superclasse e Subclasses)
  - 2: Múltiplas Relações (Apenas Subclasses)
  - 3: Relação Única com Atributo de Tipo (Especialização Disjunta)
  - 4: Relação Única com Múltiplos Atributos (Especialização Sobreposta)

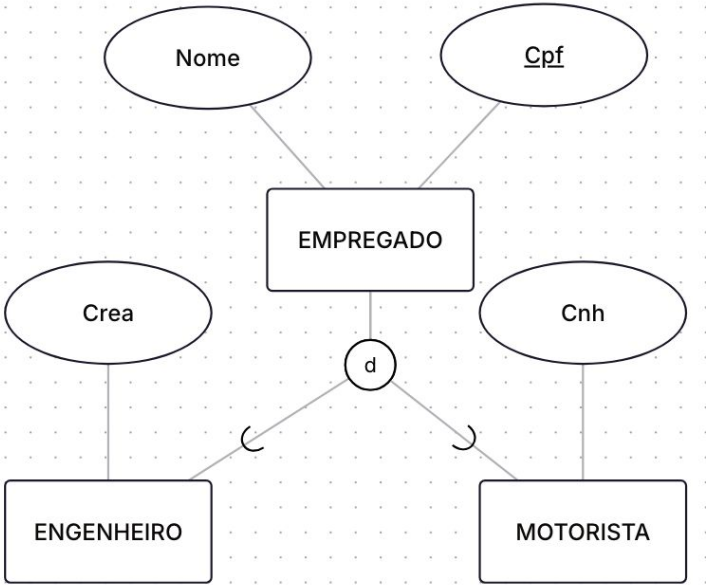
Múltiplas Relações (Superclasse e Subclasses)

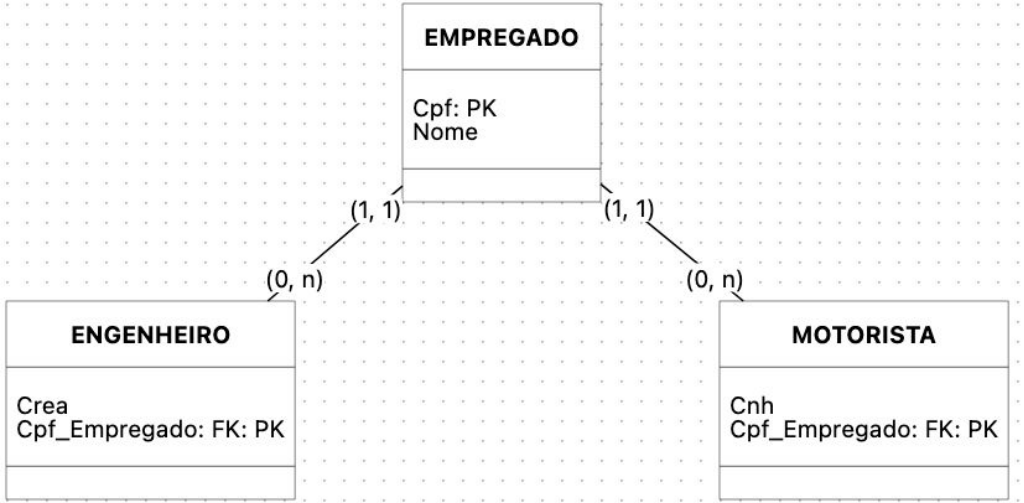
Cria-se uma tabela para a superclasse e uma tabela para cada subclasse.

As subclasses herdam a Chave Primária (PK) da superclasse, que funciona também como Chave Estrangeira (FK).

-> É a opção universal (funciona para qualquer restrição: d/o, Total/Parcial).

Múltiplas Relações (Superclasse e Subclasses)



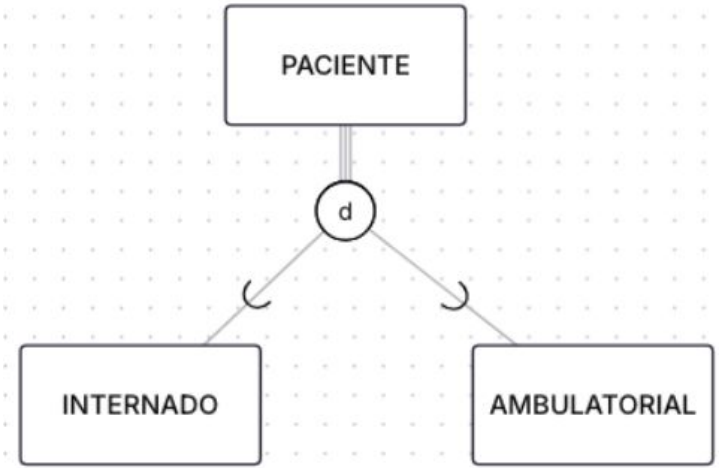


carlos.melo@upe.br

Múltiplas Relações (Apenas Subclasses)

Cria-se tabelas apenas para as subclasses. Os atributos da superclasse são replicados em cada uma delas.

Só funciona se a especialização for **Total** (todo registro obrigatoriamente pertence a uma subclasse).

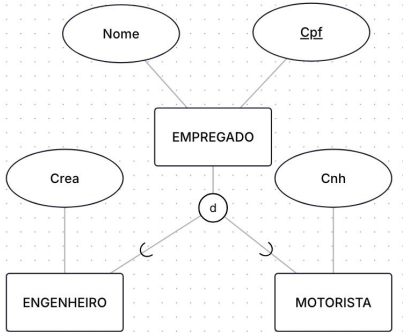


PACIENTE_AMBULATORIAL
Cpf: PK Nome

PACIENTE_INTERNADO
Cpf: PK Hora_Internacao Nome

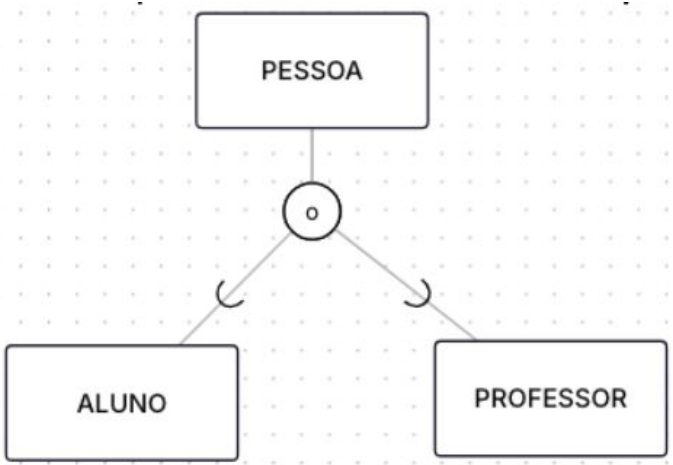
Relação Única com Atributo de Tipo (Especialização Disjunta)

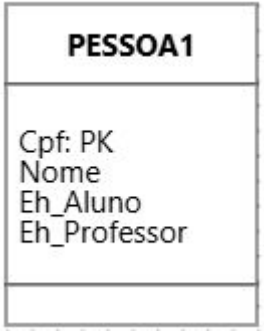
- Uma única tabela para toda a hierarquia
- Cria-se um atributo discriminador (ex: tipo_pessoa) que indica a qual subclasse o registro pertence
- Restrição:
  - Só funciona para especializações **Disjuntas (d)**



Relação Única com Múltiplos Atributos (Especialização Sobreposta)

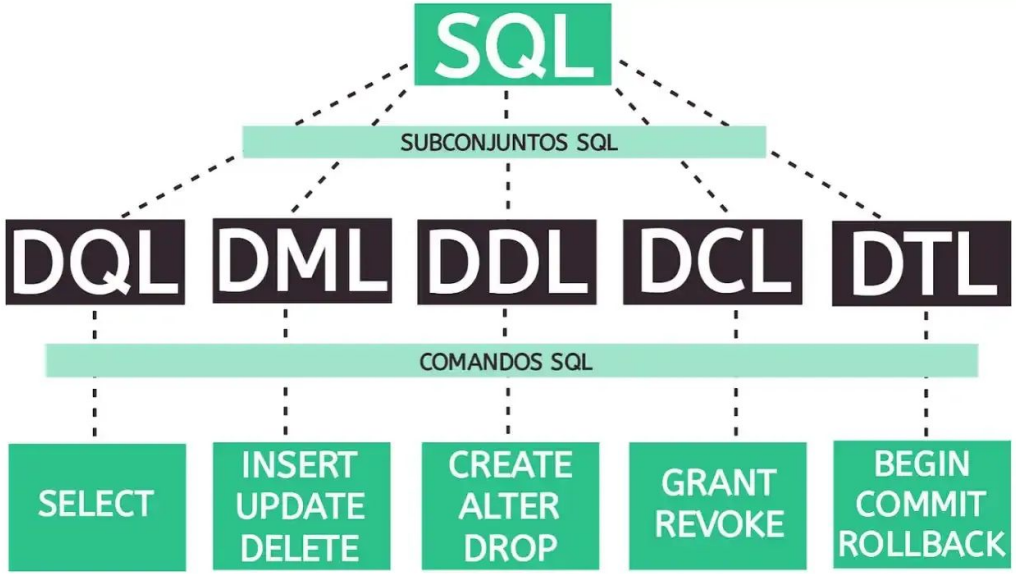
- Uma única tabela, mas com **flags** booleanas para cada subclasse (ex: eh_aluno, eh_professor)
- **Indicação** : Ideal para especializações Sobrepostas (o)





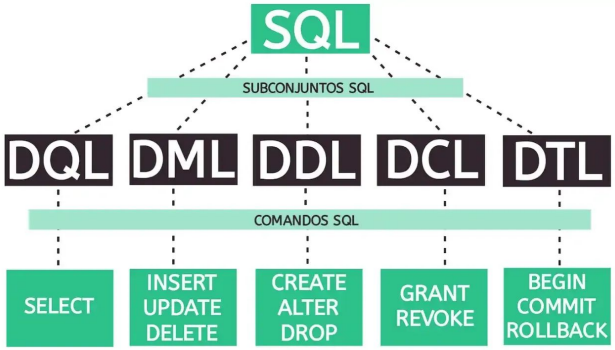
SQL

- Structure Query Language (Linguagem de Consulta Estruturada) é a linguagem padrão para operar bancos de dados relacionais.



SQL

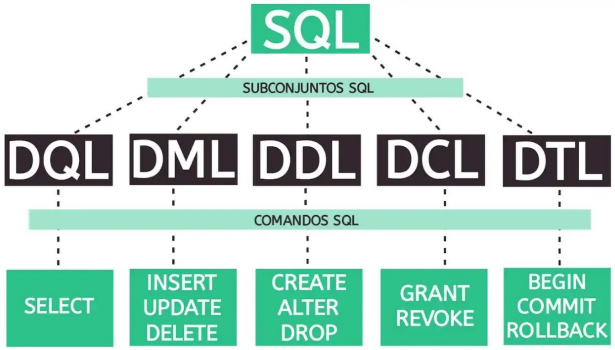
- **DDL** (Data Definition Language - Definição): Constrói a estrutura. Cria as tabelas e as regras.
  - **CREATE, ALTER, DROP**
- **DML** (Data Manipulation Language - Manipulação): Modifica os dados dentro do BD
  - **INSERT, UPDATE, DELETE**
- **DQL** (Data Query Language - Consulta): Extrai a informação.
  - **SELECT**



carlos.melo@upe.br

SQL

- **TCL ou DTL** (Data Transaction Language - Transações): Controla o "salvar" e "desfazer" das operações
  - **COMMIT, SAVEPOINT e ROLLBACK**
- **DCL** (Data Control Language - Controle): Gere permissões de acesso
  - **GRANT e REVOKE**



carlos.melo@upe.br

(SQLite)

- Diferente de PostgreSQL ou Oracle, o SQLite não precisa de um servidor de banco de dados rodando em segundo plano
- É um banco de dados inteiro salvo em um único arquivo ou diretamente em memória
- Por essas e outras é largamente utilizado em prototipagem e na Ciência de Dados
- Vamos usar o SQLite dentro do Colab antes de prosseguirmos com o PostgreSQL

(Tipos de Dados no SQLite)

- No livro são dezenas de tipos de dados apresentados `VARCHAR(50)`, `DECIMAL(10,2)`, `BOOLEAN`, etc.
- O SQLite é um banco mínimo e possui apenas 5 classes para armazenamento de dados:
  1. **NULL**: Valor ausente
  2. **INTEGER**: Números inteiros
  3. **REAL**: Números com ponto flutuante
  4. **TEXT**: Cadeias de caracteres
  5. **BLOB**: Dados binários (ex: imagens)

(Usando SQLite no Colab)

```python
import sqlite3
import pandas as pd

# Cria conexão com um banco vazio em memória
conn = sqlite3.connect(':memory:')

# Habilita Chaves Estrangeiras
conn.execute("PRAGMA foreign_keys = ON;")

# Função para executar DDL e DML (Criar e Inserir)
def executar(sql):
    conn.execute(sql)
    conn.commit()
    print("Comando executado com sucesso.")

# Função para executar DQL (Consultar)
def consultar(sql):
    return pd.read_sql_query(sql, conn)
```

DDL: CREATE TABLE

- **Regras**:
  - Definimos o nome da tabela
    - (singular + snake_case = recomendação pessoal)
  - Abrimos parênteses e listamos as colunas, seguidas dos seus tipos de dados
  - Usamos **NOT NULL** para atributos obrigatórios

```sql
CREATE TABLE departamento (
    numero INTEGER PRIMARY KEY,
    nome TEXT NOT NULL
);
```

```python
sql_criar_departamento = """
CREATE TABLE departamento (
    numero INTEGER PRIMARY KEY,
    nome TEXT NOT NULL
);
"""

executar(sql_criar_departamento)
```

carlos.melo@upe.br

DDL: Relacionamentos e **FOREIGN KEY**

- No processo de conexão definimos o uso de chave estrangeira, sem àquilo nada dará certo daqui em diante

```sql
CREATE TABLE empregado (
    cpf TEXT PRIMARY KEY,
    nome TEXT NOT NULL,
    numero_departamento INTEGER,
    FOREIGN KEY (numero_departamento) REFERENCES departamento(numero)
);
```

```python
sql_empregado = """
CREATE TABLE empregado (
    cpf TEXT PRIMARY KEY,
    nome TEXT NOT NULL,
    numero_departamento INTEGER,
    FOREIGN KEY (numero_departamento) REFERENCES departamento(numero)
);
"""
executar(sql_empregado)
```

DML: Populando as tabelas criadas

- A ordem importa (empregado precisa de um departamento)

```sql
INSERT INTO departamento (numero, nome)
VALUES (1, 'Engenharia');

INSERT INTO empregado (cpf, nome, numero_departamento)
VALUES ('123.456.789-00', 'João Silva', 1);
```

```python
executar("INSERT INTO departamento (numero, nome) VALUES (1, 'Engenharia');")

sql_insert_emp = """
INSERT INTO empregado (cpf, nome, numero_departamento)
VALUES ('123.456.789-00', 'Maria Luisa', 1);
"""

executar(sql_insert_emp)
```

carlos

DQL: SELECT _ FROM

- O comando SELECT retorna dados do banco.
- É a única operação que retorna os dados visualmente aos usuários

```
SELECT * FROM empregado;
```

```
sql_consulta = "SELECT * FROM empregado;"

df_empregados = consultar(sql_consulta)
display(df_empregados)
```

carlos.melo@upe.br

DQL: Filtros

- Na vida real não queremos trazer milhões de linhas como saída para algumas consultas, é por isso que existe o WHERE

```
SELECT nome, cpf
FROM empregado
WHERE numero_departamento = 1;
```

```
sql_filtro = """
SELECT nome, cpf
FROM empregado
WHERE numero_departamento = 1;
"""
consultar(sql_filtro)
```

carlos.melo@upe.br

Exercício

- Crie a tabela projeto. Ela deve ter um codigo (Inteiro, Chave Primária) e um nome (Texto, obrigatório).
- Crie a tabela empregado_projeto (M:N)
- Insira dois projetos novos na tabela projeto
  - (ex: Projeto 10 - 'App Mobile', Projeto 20 - 'Migração Cloud')
- Aloque o empregado João (CPF '111') no projeto 'App Mobile' (Código 10) trabalhando 40 horas
- Escreva um SELECT que mostre todos os dados da tabela empregado_projeto para confirmar que a alocação do João foi salva com sucesso.

Projeto (3pts na média final)

- Divisão dos grupos (no máx 4 pessoas por grupo)
- Aplicação real para resolver algum problema da universidade, comunidade ou do curso de vocês (30%)
  - >= 20 tabelas e/ou **views** + **triggers** + **procedures**
- Projeto completo do banco de dados **normalizado** (40%)
  - Modelo conceitual, lógico e físico
- *Seeders* para popular o banco (10%)
  - *scripts* para realização das principais consultas
- Testes a nível de aplicação e.g Python/Java (10%)
- Documentação (10%)

Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO CAMPUS GARANHUNS